

Resumen Teórico: Máquinas de Turing

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	2
2. Un poco de historia	2
2.1. Algoritmos antes de las computadoras	2
2.2. El problema con la informalidad	2
2.3. El problema décimo de Hilbert	2
2.4. Turing y la respuesta	3
3. Las Máquinas de Turing	3
3.1. Idea intuitiva	3
3.2. Definición formal	3
3.3. Funcionamiento sobre una entrada	4
4. Configuraciones y aceptación	4
4.1. Configuraciones	4
4.2. Aceptación y lenguaje reconocido	5
5. Máquinas de decisión y lenguajes decidibles	5
5.1. Relación entre decidible y reconocible	5
6. Cómo describir máquinas de Turing	6
6.1. Ejemplo 1: el lenguaje $A = \{w\#w \mid w \in \{0,1\}^*\}$	6
6.2. Ejemplo 2: el lenguaje $A = \{0^{2^n} \mid n \geq 0\}$	6
6.3. Tabla de transiciones	7
7. Variantes del modelo	7
7.1. Máquinas de Turing con múltiples cintas	7
7.2. Máquinas de Turing no deterministas	8
7.3. Otras variantes equivalentes	9
8. Equivalencia entre máquinas	9
9. Enumeradores	9
9.1. Idea intuitiva	9
9.2. Formalización	9
9.3. Lenguajes reconocibles vs. lenguajes enumerados	10
10. La Tesis de Church-Turing	10
11. Resumen final	11

1. Introducción

La idea de *algoritmo* es uno de los conceptos más antiguos de la matemática, pero su formalización es relativamente reciente. Durante miles de años, los matemáticos usaron procedimientos algorítmicos sin una definición precisa de lo que significaba “poder calcular algo”. Recién en 1936, Alan Turing propuso un modelo formal que permitió hablar con rigor matemático sobre qué problemas son **computables** y cuáles no.

Este resumen recorre el modelo de las **Máquinas de Turing** (MT): su motivación histórica, su definición formal, sus variantes (multicinta, no determinista, enumeradores), las nociones de lenguaje *reconocible* y *decidible*, y termina con la **Tesis de Church-Turing**, que sintetiza por qué este modelo se considera *la* formalización de la noción de computación.

2. Un poco de historia

2.1. Algoritmos antes de las computadoras

La idea intuitiva de algoritmo es muy anterior a la existencia de computadoras. Algunos ejemplos clásicos:

- **Tablas de arcilla de Bagdad** (aproximadamente 1600 a.C.): describen el algoritmo de la división.
- **Criba de Eratóstenes** (aproximadamente 200 a.C.): un procedimiento para encontrar los números primos menores a un valor dado.
- **Algoritmo de Euclides**: calcula el máximo común divisor de dos enteros.

2.2. El problema con la informalidad

Aunque estos algoritmos funcionaban, su descripción era *informal*. Por ejemplo, la criba de Eratóstenes se enuncia así:

Dado un N , queremos encontrar todos los primos $\leq N$:

1. Listar los números $1, 2, 3, \dots, N$.
2. Para cada $i = 2, \dots, \sqrt{N}$: tachar todos los múltiplos de i desde $i + 1$ hasta N .
3. Los números no tachados son primos.

La descripción es clara para una persona, pero **no es matemáticamente precisa**: ¿qué significa exactamente “tachar”? ¿cómo se representa la lista? Para poder *demostrar* cosas sobre algoritmos hace falta una definición formal de lo que es un algoritmo.

2.3. El problema décimo de Hilbert

En 1900, David Hilbert planteó 23 problemas que consideraba los más importantes para los matemáticos de su tiempo. El problema número 10 fue particularmente influyente:

“¿Existe un procedimiento que, en una secuencia finita de pasos, permita determinar si un polinomio tiene una raíz entera?”

Por ejemplo, dado el polinomio $6x^3yz^2 + 3xz^2$, ¿hay valores enteros de x, y, z que lo hagan cero? La pregunta de Hilbert es: ¿existe un algoritmo *general* que responda esto para cualquier polinomio?

Idea clave

Si tal procedimiento existe, alcanza con describirlo. Pero si *no* existe, demostrarlo es mucho más difícil: hay que probar que ningún algoritmo posible puede resolverlo. Y para eso primero hay que definir formalmente qué es un algoritmo.

2.4. Turing y la respuesta

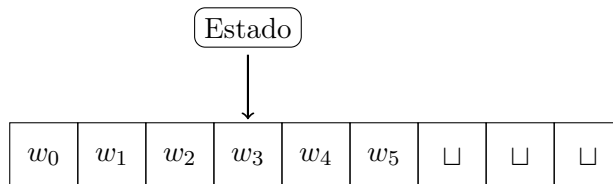
El décimo problema de Hilbert inspiró a Alan Turing a buscar una definición precisa de la noción de algoritmo, con el objetivo de poder demostrar que existen problemas **no computables**. En 1936 publicó el artículo “*On computable numbers, with an application to the Entscheidungsproblem*”, donde introdujo las máquinas que hoy llevan su nombre y demostró la existencia de problemas que ninguna máquina puede resolver.

3. Las Máquinas de Turing

3.1. Idea intuitiva

Una máquina de Turing es una formalización muy simple de la noción de computación. Consta de:

- Una **cinta** infinita dividida en celdas, donde se almacenan símbolos.
- Un **cabezal** que apunta a una celda de la cinta. Puede leer y escribir.
- Un **estado** interno (de un conjunto finito).
- Un **símbolo blanco** $\sqcup$ que aparece en las celdas vacías.



En cada paso la máquina realiza tres acciones:

1. **Lee** el símbolo bajo el cabezal.
2. Según el estado y el símbolo leído, **escribe** un nuevo símbolo.
3. **Mueve** el cabezal una posición a la izquierda (L) o a la derecha (R), y cambia su estado.

Intuición

Pensemos la cinta como una memoria ilimitada, el cabezal como un “procesador” que solo ve una celda a la vez, y los estados como las distintas situaciones internas en las que puede estar el programa. A pesar de su simplicidad, este modelo es capaz de simular cualquier computadora.

3.2. Definición formal

Definición 3.1 (Máquina de Turing). Una **máquina de Turing** (determinista) es una tupla

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_A, q_R \rangle$$

donde:

- Q es un conjunto *finito* de estados.
- Σ es el **alfabeto de entrada**, que *no* contiene el símbolo blanco $\sqcup$.
- Γ es el **alfabeto de la cinta**, con $\Sigma \subseteq \Gamma$ y $\sqcup \in \Gamma$.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ es la **función de transición**.
- $q_0 \in Q$ es el **estado inicial**.
- $q_A \in Q$ es el **estado de aceptación**.
- $q_R \in Q$ es el **estado de rechazo**, con $q_A \neq q_R$.

La transición $\delta(q, a) = (q', b, D)$ se lee así: “estando en el estado q y leyendo el símbolo a , escribo b , me muevo en la dirección $D \in \{L, R\}$ y paso al estado q' ”.

3.3. Funcionamiento sobre una entrada

Dada una cadena de entrada $w \in \Sigma^*$, la máquina M procede así:

1. Empieza en el estado q_0 , con w escrito en las primeras celdas de la cinta (el resto contiene $\sqcup$) y el cabezal apuntando al primer símbolo.
2. En cada paso aplica δ al estado y símbolo actuales: escribe, se mueve y cambia de estado.
3. Se repite hasta llegar al estado q_A (acepta) o al estado q_R (rechaza).

Hay **tres posibles resultados**:

- (I) La máquina **acepta** la entrada al llegar a q_A .
- (II) La máquina **rechaza** la entrada al llegar a q_R .
- (III) La máquina **no termina** (se queda corriendo para siempre).

Intuición

La tercera posibilidad — que la máquina no termine — es una característica fundamental del modelo. A diferencia de los autómatas finitos, una MT puede entrar en bucles infinitos. Esto, lejos de ser un “defecto”, es lo que le permite capturar la idea genuina de cómputo y, como veremos, está íntimamente relacionado con la existencia de problemas no decidibles.

4. Configuraciones y aceptación

4.1. Configuraciones

Para describir formalmente la ejecución de una MT usamos la noción de *configuración*, que captura el estado completo de la máquina en un instante.

Definición 4.1 (Configuración). Una **configuración** de una MT tiene tres componentes y se representa como

$$w q_i v$$

donde:

- $w \in \Gamma^*$ es la porción de la cinta a la izquierda del cabezal,
- $q_i \in Q$ es el estado actual,
- $v \in \Gamma^*$ es la porción de la cinta a la derecha, *incluyendo* el símbolo bajo el cabezal.

Definición 4.2 (Paso de cómputo). Escribimos $w q_i v \rightarrow_M w' q_j v'$ si la configuración $w' q_j v'$ se obtiene aplicando una vez la función de transición δ de la máquina M a la configuración $w q_i v$.

4.2. Aceptación y lenguaje reconocido

Definición 4.3 (Aceptación de una cadena). Una MT M **acepta** una cadena w si existe una secuencia de configuraciones $C_0, C_1, \dots, C_n$ tal que:

- $C_0 = q_0 w$ es la configuración inicial.
- Para cada $0 \leq i < n$, se tiene $C_i \rightarrow_M C_{i+1}$.
- C_n es una configuración de aceptación, es decir, $C_n = w' q_A w''$ para algunos $w', w'' \in \Gamma^*$.

Definición 4.4 (Lenguaje reconocido). Dada una MT M , el **lenguaje reconocido** por M es

$$\mathcal{L}(M) = \{w \in \Sigma^* \mid M \text{ acepta } w\}.$$

Definición 4.5 (Lenguaje Turing reconocible). Un lenguaje L es **Turing reconocible** (también llamado *recursivamente enumerable*) si y solo si existe una MT M tal que $\mathcal{L}(M) = L$.

Observación 4.6. Si $w \notin \mathcal{L}(M)$, la máquina puede rechazar w explícitamente (llegando a q_R) o no terminar nunca. Ambas situaciones cuentan como “ M no acepta w ”.

5. Máquinas de decisión y lenguajes decidibles

Hay un sub-tipo de MT especialmente importante: las que *siempre* terminan.

Definición 5.1 (Máquina de decisión). Una **máquina de decisión** es una MT que, para cualquier entrada, termina (en q_A o q_R). Es decir, nunca se queda corriendo indefinidamente.

Definición 5.2 (Lenguaje decidable). Un lenguaje L se dice **decidable** si y solo si existe una máquina de decisión M tal que $\mathcal{L}(M) = L$.

Idea clave

Una máquina de decisión nos permite responder con *certeza* si una cadena pertenece o no a un lenguaje. Las máquinas que no siempre terminan, en cambio, solo pueden *confirmar* pertenencia: si la cadena no está en el lenguaje, podríamos esperar para siempre sin obtener respuesta.

5.1. Relación entre decidable y reconocible

Toda máquina de decisión es, en particular, una MT, así que:

$$\text{decidable} \implies \text{Turing reconocible}.$$

Pero la implicación inversa no vale. Existen lenguajes que son Turing reconocibles *pero no decidibles*: una MT los reconoce, pero no hay forma de garantizar que termine cuando la cadena no pertenece al lenguaje.

Ejemplo 5.3 (Reconocible pero no decidable). Consideremos

$$L = \{\phi \mid \vdash \phi\}$$

el lenguaje de las fórmulas de primer orden que son válidas (es decir, los teoremas demostrables).

Podemos pensar una MT que reconozca L del siguiente modo:

1. Empieza con la fórmula ϕ codificada en la cinta.
2. Detrás de ϕ escribe un separador $\#$.
3. Va enumerando *todas* las posibles demostraciones (combinando reglas y axiomas).
4. Si en algún momento la demostración produce ϕ , acepta.

Si ϕ es demostrable, eventualmente aparecerá entre las demostraciones generadas y la máquina aceptará. Pero si ϕ *no* es demostrable, la máquina sigue buscando para siempre. Puede probarse que este lenguaje *no* es decidible: no existe ningún algoritmo que siempre termine y decida correctamente si una fórmula es teorema.

6. Cómo describir máquinas de Turing

Para diseñar MTs usamos distintos niveles de descripción: *alto nivel* (en palabras), *diagrama de estados*, o *tabla de transiciones*.

6.1. Ejemplo 1: el lenguaje $A = \{w\#w \mid w \in \{0,1\}^*\}$

Queremos una MT que reconozca cadenas de la forma $w\#w$.

Idea de alto nivel: comparar carácter a carácter, marcando cada símbolo ya comparado con una x para no volver a procesarlo.

Algoritmo:

1. Dado un símbolo b a la izquierda del $\#$, reemplazarlo por x y avanzar a la derecha hasta encontrar el primer símbolo después del $\#$ que no sea x .
 - Si ese símbolo es igual a b , reemplazarlo por x y volver al comienzo.
 - Si es distinto (o no hay), rechazar.
2. Cuando se llegue a una configuración donde antes del $\#$ todo es x , verificar que después del $\#$ también es todo x . Si es así, aceptar; sino, rechazar.

Definición formal: con $\Sigma = \{0,1,\#\}$ y $\Gamma = \{0,1,\#,x,\sqcup\}$, la máquina M_1 tiene estados $q_1, \dots, q_8, q_{\text{accept}}$ (más q_R implícito) y un diagrama que coordina:

- q_1 : leer el primer símbolo sin marcar de la izquierda y marcarlo con x .
- q_2, q_3 : recordar si el símbolo borrado era 0 o 1, mientras se cruza hacia la derecha del $\#$.
- q_4, q_5 : buscar el primer símbolo no marcado a la derecha y compararlo.
- q_6, q_7 : regresar a la izquierda para repetir el proceso.
- q_8 : verificar que después de $\#$ todo sea x , y aceptar.

Las transiciones *no especificadas* explícitamente se asume que van al estado de rechazo q_R .

6.2. Ejemplo 2: el lenguaje $A = \{0^{2^n} \mid n \geq 0\}$

Aquí queremos reconocer cadenas con 1, 2, 4, 8, 16, ... ceros (potencias de 2).

Idea de alto nivel: en cada pasada por la cinta, “tachar” la mitad de los ceros. Si en alguna pasada queda exactamente un cero, aceptamos. Si en alguna pasada queda una cantidad impar mayor a 1, rechazamos (porque la cantidad no era potencia de 2).

Algoritmo:

1. Reemplazar el primer 0 por $\sqcup$. Si no había más símbolos, *aceptar*.
2. Si hay otro 0, reemplazarlo por x (esta primera “marca” indica el inicio de la pasada).
3. Para cada 0 a la derecha, reemplazar uno sí y uno no por x (es decir, contar de a pares).
4. Si la cantidad de ceros era impar (mayor a 1), rechazar.
5. Al llegar al final, volver al principio y repetir el paso 3 sobre los ceros restantes.
6. Si en algún momento solo queda un cero, aceptar.

Cada pasada divide a la mitad la cantidad de ceros sin marcar; si la cantidad original era 2^n , después de n pasadas queda 1 y aceptamos. Si no era potencia de 2, en alguna pasada la cantidad será impar y rechazamos.

6.3. Tabla de transiciones

También se puede definir δ mediante una **tabla**: para cada par (estado, símbolo), se indica el símbolo a escribir, el movimiento (L/R) y el estado siguiente.

Observación práctica: para máquinas grandes, tanto el diagrama como la tabla resultan *impracticables*. Por eso normalmente diseñamos las MTs con descripciones de alto nivel.

7. Variantes del modelo

Una propiedad muy importante del modelo de Turing es su **robustez**: muchas variantes naturales (más cintas, no determinismo, cinta doblemente infinita, etc.) resultan equivalentes en poder de cómputo a la MT original. Esto es una evidencia fuerte a favor de que las MTs capturan la noción intuitiva de cómputo.

7.1. Máquinas de Turing con múltiples cintas

Una MT con k cintas es como la versión estándar pero con k cintas independientes, cada una con su propio cabezal. La cadena de entrada está inicialmente en la primera cinta; las otras empiezan vacías.

Definición 7.1 (MT multicinta). Una MT con k cintas es una tupla

$$\langle Q, \Sigma, \{\Gamma_i\}_{0 \leq i < k}, \delta, q_0, q_A, q_R \rangle$$

donde Γ_i es el alfabeto de la i -ésima cinta y

$$\delta : Q \times \Gamma_0 \times \cdots \times \Gamma_{k-1} \longrightarrow Q \times \Gamma_0 \times \cdots \times \Gamma_{k-1} \times \{L, R\}^k.$$

En cada paso, la máquina lee *los k símbolos* bajo los cabezales, escribe en cada cinta, y mueve cada cabezal en su propia dirección.

Teorema 7.2 (Equivalencia multicinta–una cinta). Para toda MT con k cintas existe una MT con una sola cinta equivalente: reconocen el mismo lenguaje.

Idea de la prueba (simulación). Dada una MT M con k cintas, construimos una MT S con una sola cinta que simula a M :

- En la cinta de S se concatenan los contenidos de las k cintas de M , separados por el símbolo $\#$:

$$\# t_1 \# t_2 \# \cdots \# t_k \#$$

- Para indicar dónde está el cabezal de cada cinta, se introducen **símbolos marcados** (por ejemplo, $\bar{a}$ representa el símbolo a con el cabezal encima).
- Para simular un paso de M , la máquina S recorre la cinta para localizar los k símbolos marcados, decide qué hacer según δ , y luego vuelve a recorrer la cinta para realizar las modificaciones y mover los cabezales virtuales.
- Si una cinta virtual necesita más espacio (por ejemplo, su cabezal se “sale” del segmento asignado), S desplaza todo lo que está a la derecha una posición.

Esta simulación tiene un costo en tiempo (es *cuadráticamente* más lenta), pero reconoce exactamente el mismo lenguaje.

7.2. Máquinas de Turing no deterministas

En una MT **no determinista** (NTM, por *nondeterministic Turing machine*), la transición ya no produce *un* sucesor sino un *conjunto* de sucesores posibles. Es decir, desde una misma configuración la máquina puede “ramificarse” en varias ejecuciones.

Definición 7.3 (MT no determinista). Una MT no determinista difiere de la determinista solo en la función de transición:

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

Es decir, $\delta(q, a)$ es un *conjunto* de tripletas, posiblemente con más de un elemento.

Propiedades importantes:

- Cada configuración tiene una *cantidad finita* de posibles sucesores.
- La ejecución forma un **árbol** (no una secuencia lineal).
- Se dice que la máquina **acepta** la cadena si *existe al menos una rama* del árbol que llegue a q_A .
- No es un modelo realista de cómputo: no se corresponde con una computadora real, pero es una herramienta teórica muy útil.

Intuición

Podemos pensar al no determinismo como un “adivino mágico” que siempre toma la decisión correcta. Si *alguna* secuencia de decisiones lleva a aceptar, la máquina acepta. Equivalentemente: se ejecutan todas las ramas en paralelo y basta con que una acepte.

Teorema 7.4 (Equivalencia DTM–NTM). Para toda MT no determinista N existe una MT determinista M equivalente, es decir, $\mathcal{L}(N) = \mathcal{L}(M)$.

Idea de la prueba. La idea es recorrer el árbol de ejecución de N con un algoritmo de búsqueda en anchura (**BFS**):

- M explora todas las ramas del árbol de ejecución de N , nivel por nivel.
- Si *alguna* rama acepta la cadena, M acepta.
- Es importante usar BFS y no DFS: si usáramos DFS y la primera rama no terminara, nunca exploraríamos las otras ramas (que podrían aceptar). Con BFS, si hay una rama que acepta a profundidad finita, en algún momento la encontramos.
- Si N no acepta la cadena y alguna rama no termina, entonces M tampoco terminará.

Conclusión: el no determinismo no agrega poder de cómputo, solo cambia la *eficiencia* (y de hecho, la relación entre clases deterministas y no deterministas es uno de los problemas abiertos centrales en teoría de la complejidad: P vs. NP).

7.3. Otras variantes equivalentes

Otras variantes que también resultan equivalentes a la MT estándar (no se demuestra en detalle aquí, pero la idea es similar):

- **Cinta doblemente infinita:** la cinta es infinita tanto a izquierda como a derecha. Se simula con una cinta normal “doblando” la cinta sobre sí misma y usando dos pistas.
- Diferentes alfabetos, números variables de estados, etc.

8. Equivalencia entre máquinas

Definición 8.1 (Máquinas equivalentes). Dos MTs M y M' son **equivalentes** si para toda cadena w :

- M acepta w si y solo si M' acepta w ;
- M rechaza w si y solo si M' rechaza w ;
- M no termina con w si y solo si M' no termina con w .

Observación 8.2. La **equivalencia** entre MTs es un problema **no decidible**: no existe ningún algoritmo que, dadas dos MTs cualesquiera, siempre termine y decida si son equivalentes. Este es uno de los resultados de imposibilidad clásicos de la teoría de la computabilidad.

9. Enumeradores

9.1. Idea intuitiva

Hasta ahora pensamos a las MTs como “reconocedoras”: les damos una cadena y nos dicen si la aceptan o no. Otra perspectiva igualmente útil es la de máquinas que *producen* todas las cadenas de un lenguaje, una tras otra. A estas máquinas las llamamos **enumeradores**.

Un enumerador, intuitivamente, es una MT con una “impresora”: cada cierto tiempo escribe una cadena en su salida. El conjunto de todas las cadenas que llega a imprimir constituye el lenguaje *enumerado* por la máquina.

9.2. Formalización

Definición 9.1 (Enumerador). Un **enumerador** es una MT con **dos cintas**:

$$\langle Q, \Sigma, \Gamma_0, \Gamma_1, q_0, q_{\text{init}}, q_A, q_R, q_{\text{print}} \rangle$$

donde:

- Γ_0 es el alfabeto de la cinta de trabajo.
- Γ_1 es el alfabeto de la cinta de impresión (output).
- q_{print} es un **estado de impresión**: cada vez que se llega a q_{print} , se “emite” el contenido actual de la cinta de impresión y luego se limpia.

Definición 9.2 (Lenguaje enumerado). Un lenguaje L es **recursivamente enumerado** (por un enumerador E) si existe un enumerador E tal que el conjunto de todas las cadenas que E imprime es exactamente L .

9.3. Lenguajes reconocibles vs. lenguajes enumerados

Los conceptos de Turing reconocible y enumerable por un enumerador coinciden:

Teorema 9.3. Un lenguaje L es Turing reconocible si y solo si existe un enumerador E que lo enumera.

Prueba. La demostración va en ambos sentidos.

($\Leftarrow$) **De enumerador a reconocedor.** Supongamos que E enumera L . Construimos una MT M con dos cintas que reconoce L :

- Dada una entrada w , M simula E en la segunda cinta.
- Cada vez que E imprime una cadena w' , M la compara con w .
- Si en algún momento $w' = w$, la máquina M acepta.
- Si $w \notin L$, E no imprime w y M nunca acepta (no termina o sigue esperando).

($\Rightarrow$) **De reconocedor a enumerador.** Dada una MT M que reconoce L , construimos un enumerador E así:

- E ignora su entrada.
- Sea $s_1, s_2, s_3, \dots$ una enumeración (computable) de todas las cadenas de Σ^* .
- Para cada $i = 1, 2, 3, \dots$:
 - Correr M durante i pasos sobre cada una de las cadenas $s_1, s_2, \dots, s_i$.
 - Cada vez que M acepta alguna de estas cadenas dentro de i pasos, imprimirla.

¿Por qué funciona? Si $w \in L$, entonces M acepta w en algún número finito de pasos k . Cuando $i \geq \max(k, j)$ (donde $s_j = w$), la doble pasada del bucle llega a probar M con w por i pasos, encuentra la aceptación e imprime w . Por lo tanto, toda cadena de L termina siendo impresa.

Idea clave

La técnica de simular “ M por i pasos sobre las primeras i cadenas, para $i = 1, 2, 3, \dots$ ” se llama **dovetailing** y es fundamental. Permite “ejecutar en paralelo” infinitas computaciones sin quedarse atrapado en una sola que nunca termina.

10. La Tesis de Church-Turing

A lo largo de la historia se propusieron diferentes formalizaciones de la idea de computación: las MTs de Turing, el λ -cálculo de Church, las funciones recursivas de Gödel-Kleene, los sistemas de Post, las máquinas RAM, los lenguajes de programación modernos, etc.

Hecho notable: *todas estas formalizaciones resultaron ser equivalentes entre sí.* Cualquier función calculable por uno de estos modelos también lo es por los demás.

Tesis de Church-Turing

La noción intuitiva de *algoritmo* (o *función computable*) coincide con la noción formal de *función computable por una máquina de Turing*. En otras palabras: *todos los modelos razonables de computación son equivalentes en poder a la máquina de Turing*.

Importante: la tesis de Church-Turing *no es un teorema*: no se puede demostrar matemáticamente, porque la idea informal de algoritmo no está formalizada. Es una afirmación *filosófica/empírica*, sustentada en la enorme evidencia acumulada de que todos los modelos propuestos coinciden.

11. Resumen final

Las ideas centrales que conviene tener clarísimas:

1. **Una MT es una formalización muy simple de la noción de cómputo:** una cinta infinita, un cabezal con un estado, y una función de transición que dice qué hacer en cada paso.
2. **Una MT tiene tres comportamientos posibles ante una entrada:** aceptar, rechazar o no terminar.
3. **Distinguir dos clases de lenguajes es fundamental:**
 - **Decidibles:** existe una MT que siempre termina y reconoce el lenguaje.
 - **Turing reconocibles:** existe una MT que reconoce el lenguaje (pero puede no terminar sobre cadenas que no pertenecen).

Todo decidable es reconocible, pero no al revés.

4. **El modelo es muy robusto.** Variantes naturales — multicinta, no determinista, cinta doblemente infinita — son todas *equivalentes* en poder de cómputo a la MT estándar. Pueden cambiar la eficiencia, pero no qué lenguajes pueden reconocer.
5. **Los lenguajes Turing reconocibles coinciden con los lenguajes enumerables** por un enumerador. La doble caracterización (reconocedor / enumerador) es muy útil para demostrar propiedades.
6. **Existen problemas no decidibles**, e incluso problemas Turing reconocibles que no son decidibles (como la validez de fórmulas de primer orden, o la equivalencia entre dos MTs).
7. **Tesis de Church-Turing:** todos los modelos razonables de cómputo son equivalentes en poder a la MT. Esto justifica usar las MTs como modelo canónico de “lo que se puede computar”.

Intuición

Si en algún momento te preguntás “¿este problema se puede resolver con una computadora?”, la respuesta formal es: “ $\Leftrightarrow$ existe una MT que lo decide”. Y si tenés una idea informal de un algoritmo, por la tesis de Church-Turing tenés implícita la garantía de que se puede convertir en una MT. Por eso, en el resto de la materia, vamos a poder dar algoritmos en lenguaje informal sin perder rigor.